

KREIS · GUIA PARA COLABORADORES

Repo Handbook

Contexto practico para modificar este repositorio sin romper la arquitectura. Kreis es una PWA React/Vite con una API Express/Node.js organizada como monolito modular.

1. Mapa del proyecto

apps/	
web/	Frontend PWA React/Vite
api/	Backend Express/Node.js
packages/	
shared/	Contratos TypeScript compartidos
infra/	
database/	Schema, migraciones, seeds y fixtures
docs/	
architecture/	Decisiones tecnicas y ownership
design/	Direccion visual
product/	Contexto funcional
onboarding/	Esta guia y material para colaboradores
codex-skills/	Skills locales para agentes/Codex

2. Reglas de oro

1. No agregues archivos sueltos en la raiz salvo configs del repo.
2. Si pertenece al frontend, va en `apps/web`.
3. Si pertenece al backend, va en `apps/api`.
4. Si lo usan frontend y backend, va en `packages/shared`.
5. Si es schema, migracion, seed o fixture, va en `infra/database`.
6. Si cambia una decision tecnica, documentala en `docs/architecture`.
7. No instales dependencias "por las dudas".
8. Antes de pedir review, corre lint, typecheck, test y build.

3. Comandos obligatorios

```
npm.cmd run dev
npm.cmd run dev:api
npm.cmd run lint
npm.cmd run test
npm.cmd run typecheck
npm.cmd run typecheck:api
npm.cmd run build
```

Para formato: `npm.cmd run format:check` revisa y `npm.cmd run format` aplica cambios. Evita meter un diff enorme solo de formato si tu cambio era chico.

4. Frontend PWA

```
apps/web/
  index.html
  public/
    manifest.json
    icons/
  src/
    app/
    components/
    data/
    pwa/
    styles/
    test/
    utils/
```

- Componentes React en `apps/web/src/components`.
- Estilos globales en `apps/web/src/styles/global.css`.
- Utilidades puras en `apps/web/src/utils`.
- Manifest e iconos instalables en `apps/web/public`.
- La configuracion PWA vive en `apps/web/vite.config.ts`.

- No tocar `dist/sw.js`: se genera automaticamente.

5. Backend API

```
apps/api/src/  
  platform/      HTTP server, middlewares, route registry  
  core/          config, errores, primitives compartidas  
  shared/        helpers internos del backend  
  modules/       Dominios del producto
```

Cada modulo debe usar:

```
module-name/  
  api/           Rutas, controladores, DTOs HTTP  
  application/   Casos de uso y orquestacion  
  domain/        Entidades, reglas y value objects  
  infrastructure/ Repositorios, queries, adaptadores externos  
  tests/         Tests del modulo  
  README.md      Ownership y limites del modulo
```

- `api` valida request/response y delega.
- `domain` no depende de Express ni de la base de datos.
- Un modulo no importa `infrastructure` de otro modulo.
- Validar entradas con `zod` cuando haya endpoints reales.
- No mezclar decisiones de DB en controladores.

6. Shared y database

packages/shared

Contratos de API, DTOs, enums y schemas `zod` compartidos. No poner React, Express ni queries.

infra/database

Schema, migraciones, seeds y fixtures. Toda modificacion estructural debe pasar por migracion.

7. Dependencias

- Verifica primero si ya hay algo en el repo.
- Prefiere librerias conocidas y mantenidas.
- Instala con version exacta cuando importe reproducibilidad.
- Revisa `package-lock.json`.
- Corre lint, test, typecheck y build despues de instalar.

8. Checklist antes de PR

- ☐ El cambio esta en la carpeta correcta.
- ☐ No agregue archivos sueltos en raiz.
- ☐ No rompi boundaries entre frontend/backend/shared/infra.
- ☐ Actualice docs si cambie estructura, comandos, contratos o endpoints.
- ☐ No instale dependencias innecesarias.
- ☐ Corri `npm.cmd run lint`.
- ☐ Corri `npm.cmd run test`.
- ☐ Corri `npm.cmd run typecheck`.
- ☐ Corri `npm.cmd run build`.
- ☐ Si toque backend, corri `npm.cmd run typecheck:api`.

9. Frenar y consultar si...

- Vas a cambiar carpetas de alto nivel.

- Necesitas mover contratos entre frontend y backend.
- Queres instalar una dependencia grande.
- Vas a cambiar `vite.config.ts`, `tsconfig`, `package.json` o `vercel.json`.
- Vas a tocar migraciones o schema sin decision de DB documentada.
- Estas copiando logica entre modulos en vez de ubicar un owner.